Step 4: Sending Messages from Server

Server

Now, we implement socketServer's conversationMessageHandler function; this function gets the `message` and `IDs` from client, makes the `aiMessage` and returns a new conversation to the client.

src/socketServer.js

conversationMessageHandler function

In this function, `sessionId, message, conversationId` are given as arguments.

```
const conversationMessageHandler = (socket, data) => {
  const { sessionId, message, conversationId } = data;
```

First, we make sure that the session exists. (If the server was restarted, the old `sessionId` is unknown; without this line, the client would wait forever for an answer.) Then, from the `sessionId`, we find the conversation with `conversationId`.

```
  if (!sessions[sessionId]) sessions[sessionId] = [];

  const aiMessage = {
    content: "Hello here is AI",
    id: uuid(),
    aiMessage: true,
  };

  const conversation = sessions[sessionId].find(
    (c) => c.id === conversationId
  );
```

For now, we don't use ChatGPT API, so we use "Hello here is AI" to mimic ChatGPT output.

If the conversation is not found, we create a new conversation; otherwise, we append the new messages to the existing conversation.

```
  if (!conversation) {
    sessions[sessionId].push({
      id: conversationId,
      messages: [message, aiMessage],
    });
  } else {
    conversation.messages.push(message, aiMessage);
  }
```

Then emit the updated conversation using the `conversation-details` API to the client.

```
  const updatedConversation = sessions[sessionId].find(
    (c) => c.id === conversationId
  );

  socket.emit("conversation-details", updatedConversation);
};
```

conversationDeleteHandler function

This function deletes the conversations that are attached to `sessionId`.

```
const conversationDeleteHandler = (_, data) => {
  const { sessionId } = data;

  if (sessions[sessionId]) {
    sessions[sessionId] = [];
  }
};

module.exports = { registerSocketServer };
```

Client

We need to add two reducers and actions:

Dashboard/dashboardSlice.js

The `setConversationHistory` reducer finds the conversation with `id` and updates its messages with the received messages; if it is not found, we create a new one.

```
setConversationHistory: (state, action) => {
  const { id, messages } = action.payload;
  const conversation = state.conversations.find((c) => c.id === id);
  if (conversation) {
    conversation.messages = messages;
  } else {
    state.conversations.push({ id, messages });
  }
},
```

Using this reducer, we can update the messages of a

The `deleteConversations` reducer removes the conversations and sets `selectedConversationId` to null.

```
    deleteConversations: (state) => {
      state.conversations = [];
      state.selectedConversationId = null;
    },
```

socketConnection/socketConn.js

The `conversation-details` WebSocket API is processed in this module.

```
socket.on("conversation-details", (conversation) => {
  store.dispatch(setConversationHistory(conversation));
});
```

The received `conversation` is stored in the state field using the `setConversationHistory` action.

We need to update the `sendConversationMessage` and `deleteConversations`, as we get the current `sessionId` from localStorage.

```
export const sendConversationMessage = (message, conversationId) => {
  socket.emit("conversation-message", {
    sessionId: localStorage.getItem("sessionId"),
    message,
    conversationId,
  });
};

export const deleteConversations = () => {
  socket.emit("conversation-delete", {
    sessionId: localStorage.getItem("sessionId"),
  });
};
```

Dashboard/Sidebar/DeleteConversationsButton.js

When the button is clicked, the conversations are removed from the state field (client) and from the sessions (server).

The reducer is renamed on import to avoid a name conflict with the `deleteConversations` function of `socketConn.js`.

```
import { deleteConversations as deleteConversationsFromStore } from "../dashboardSlice";
import { deleteConversations } from "../../socketConnection/socketConn";

const DeleteConversationsButton = () => {
  const dispatch = useDispatch();

  const handleDeleteConversations = () => {
    dispatch(deleteConversationsFromStore()); // client: state field
    deleteConversations(); // server: conversation-delete API
  };
  ...
```